

DOKUMEN TUGAS BESAR

IF2224 Teori Bahasa Formal dan Automata

Arion Compiler : Milestone 4 Intermediate Code Generator & Interpreter



Nama Anggota :

Zahran Alvan Putra Winarko	13524124
Neswa Eka Anggara	13524136
Daniel Putra Rywandi S.	13524143
Muh. Hartawan Haidir	13524147

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA - TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025/2026**

BAB I

TEORI SINGKAT

1. Intermediate Code dan Three-Address Code (TAC)

Intermediate Code merupakan representasi perantara yang dihasilkan oleh kompiler setelah tahap Semantic Analysis dan sebelum tahap Code Generation akhir. Representasi ini bersifat platform-independent sehingga memudahkan portabilitas dan optimasi kode sebelum diterjemahkan ke kode mesin. Salah satu bentuk intermediate code yang paling umum digunakan adalah Three-Address Code (TAC).

Three-Address Code adalah representasi instruksi sekuensial berbasis stack di mana setiap instruksi memiliki paling banyak tiga operand: satu operator, maksimal dua operand sumber, dan satu operand tujuan. Setiap instruksi TAC bersifat atomik dan merepresentasikan satu operasi tunggal. Format umum instruksi TAC adalah:

$$x := y \text{ op } z$$

di mana x adalah hasil, y dan z adalah operand, serta op adalah operator. Dalam konteks Arion Compiler, TAC diimplementasikan sebagai daftar instruksi flat yang masing-masing terdiri dari opcode, level leksikal, dan satu argumen integer.

TAC memiliki beberapa keunggulan dibandingkan representasi lain. Pertama, struktur linearnya memudahkan implementasi interpreter dan analisis alir data. Kedua, instruksi berbasis stack meminimalkan kebutuhan register sementara. Ketiga, TAC cukup ekspresif untuk merepresentasikan seluruh konstruk bahasa Arion mulai dari ekspresi sederhana hingga pemanggilan fungsi bersarang.

2. Intermediate Code Generator (ICG)

Intermediate Code Generator merupakan komponen kompiler yang bertanggung jawab mengubah Decorated Abstract Syntax Tree (AST) hasil Milestone 3 menjadi daftar instruksi TAC. Proses konversi ini dilakukan melalui penelusuran pohon AST secara DFS-based (Depth-First Search-based), yaitu pendekatan traversal yang mengunjungi setiap node secara rekursif dari root hingga ke leaf.

Pendekatan yang digunakan dalam ICG Arion Compiler adalah pendekatan berbasis Visitor Pattern, di mana setiap jenis node AST memiliki fungsi generator tersendiri. Fungsi-fungsi ini dipanggil secara rekursif sesuai dengan struktur pohon. Setiap fungsi generator mengeluarkan satu atau lebih instruksi TAC ke dalam daftar instruksi yang terus berkembang selama proses traversal berlangsung.

Salah satu mekanisme penting dalam ICG adalah backpatching. Backpatching diperlukan karena pada saat instruksi jump (JMP atau JPC) pertama kali digenerate, alamat tujuan lompatan belum diketahui. Alamat tersebut baru dapat ditentukan setelah bagian kode yang menjadi tujuan lompatan selesai digenerate. Backpatching menyelesaikan masalah ini dengan mencatat indeks instruksi yang perlu diperbarui, kemudian mengisi alamat tujuan yang tepat ketika sudah diketahui.

3. Stack Machine

Stack Machine adalah model komputasi yang menggunakan struktur data tumpukan (stack) sebagai media utama untuk menyimpan data dan hasil komputasi sementara. Berbeda dengan register machine yang menggunakan register CPU untuk menyimpan nilai, stack machine mendorong (push) dan menarik (pop) nilai dari stack untuk setiap operasi.

Model stack machine memiliki karakteristik berikut. Pertama, setiap operasi aritmatika dan logika mengambil operand dari puncak stack dan mendorong hasilnya kembali ke stack. Kedua, instruksi LOAD membaca nilai dari memori dan mendorongnya ke stack, sedangkan instruksi STORE menarik nilai dari stack dan menyimpannya ke memori. Ketiga, pemanggilan fungsi atau prosedur mengalokasikan frame baru di atas stack yang berisi tautan-tautan kontrol dan variabel lokal.

Stack Machine yang diimplementasikan dalam Arion Compiler menggunakan model memori flat berupa array satu dimensi. Setiap pemanggilan prosedur atau fungsi mengalokasikan sebuah frame dengan layout sebagai berikut:

Slot	Nama	Keterangan
base+0	Static Link	Tautkan ke frame induk (parent lexical)
base+1	Dynamic Link	Nilai stack pointer caller sebelum frame ini
base+2	Return Address	Nilai program counter setelah

		instruksi CAL
base+3 dst	Variabel Lokal	Variabel-variabel lokal mulai dari offset 0

Tiga slot pertama dari setiap frame (Static Link, Dynamic Link, Return Address) bersifat reserved dan dikelola secara otomatis oleh instruksi CAL dan INT. Oleh karena itu, sebuah program Arion dengan n variabel lokal akan menghasilkan instruksi INT 0 ($3+n$) untuk mengalokasikan frame yang tepat.

4. Siklus Eksekusi (Fetch — Decode — Execute)

Interpreter pada Stack Machine mengikuti siklus eksekusi klasik yang terdiri dari tiga tahap berulang, yaitu Fetch, Decode, dan Execute.

Tahap Fetch adalah pengambilan instruksi dari daftar instruksi TAC berdasarkan nilai program counter (pc) saat ini. Setelah instruksi berhasil diambil, nilai pc dinaikkan satu untuk menunjuk ke instruksi berikutnya.

Tahap Decode adalah identifikasi jenis instruksi (opcode) dan argumennya. Berdasarkan opcode yang ditemukan, interpreter menentukan handler yang tepat untuk dieksekusi.

Tahap Execute adalah pelaksanaan operasi yang sesuai dengan instruksi yang telah di-decode. Tahap ini dapat melibatkan operasi stack (push/pop), akses memori (load/store), perubahan alur kontrol (jump), atau pemanggilan sistem (output).

Siklus ini terus berulang selama program counter valid dan program belum mencapai kondisi berhenti, yaitu ketika instruksi RET dieksekusi oleh frame program utama.

5. Set Instruksi TAC Arion Compiler

Arion Compiler mendefinisikan sembilan instruksi TAC beserta empat belas operasi aritmatika, logika, dan I/O yang diakses melalui instruksi OPR. Berikut adalah penjelasan masing-masing instruksi:

Instruksi	Format	Keterangan
INT	INT 0 m	Alokasi frame memori ukuran m. Mengisi m slot dengan nilai 0 dan mencatat base pointer ke bpStack.
LIT	LIT 0 v	Push literal v ke puncak stack.

LOD	LOD 0 a	Load nilai dari alamat absolut a pada stack, lalu push ke puncak stack.
STO	STO 0 a	Pop nilai dari puncak stack, lalu simpan ke alamat absolut a.
OPR	OPR 0 o	Jalankan operasi ke-o (lihat Tabel Operasi OPR).
JMP	JMP 0 l	Unconditional jump: set program counter ke baris l tanpa kondisi.
JPC	JPC 0 l	Conditional jump: pop nilai dari stack; jika nilainya false (0), lompat ke baris l.
CAL	CAL 0 l	Simpan return address dan dynamic link, lalu lompat ke baris l untuk mengeksekusi fungsi atau prosedur.
RET	RET 0 0	Return dari fungsi atau prosedur: pulihkan stack pointer dan program counter dari frame saat ini.

Operasi yang tersedia melalui instruksi OPR adalah sebagai berikut:

No.	Nama	Keterangan
1	NEG	Negasi unary: pop a, push -a.
2	ADD	Penjumlahan: pop b dan a, push a+b.
3	SUB	Pengurangan: pop b dan a, push a-b.
4	MUL	Perkalian: pop b dan a, push a*b.
5	DIV	Pembagian integer: pop b dan a, push a div b. Melempar error jika b=0.
6	MOD	Modulus: pop b dan a, push a mod b. Melempar error jika b=0.
7	EQL	Kesamaan: pop b dan a, push 1 jika a=b, push 0 jika tidak.
8	NEQ	Ketidaksamaan: pop b dan a, push 1 jika $a \neq b$, push 0 jika tidak.
9	LSS	Kurang dari: pop b dan a, push 1 jika $a < b$, push 0 jika tidak.
10	GEQ	Lebih dari atau sama dengan: pop b dan a, push 1 jika $a \geq b$.
11	GTR	Lebih dari: pop b dan a, push 1 jika $a > b$, push 0 jika tidak.

12	LEQ	Kurang dari atau sama dengan: pop b dan a, push 1 jika $a \leq b$.
13	WRT	Write: pop nilai dari stack dan cetak ke output tanpa newline.
14	WRTLN	Writeln: pop nilai dari stack dan cetak ke output dengan newline.

6. Generasi Kode untuk Control Flow

Konstruksi control flow seperti IF-ELSE dan WHILE memerlukan mekanisme khusus dalam pembangunan TAC karena melibatkan lompatan ke baris yang belum diketahui pada saat instruksi jump pertama kali digenerate. Berikut adalah pola generasi TAC untuk masing-masing konstruksi:

IF-ELSE

Ekspresi kondisi dievaluasi terlebih dahulu dan hasilnya dipush ke stack. Instruksi JPC kemudian digenerate dengan alamat tujuan yang masih kosong (placeholder). Blok THEN digenerate, diikuti instruksi JMP untuk melompati blok ELSE. Setelah posisi blok ELSE diketahui, alamat JPC di-backpatch. Blok ELSE digenerate, dan setelah selesai, alamat JMP di-backpatch ke baris setelah ELSE.

WHILE

Label awal loop (WHILE_START) dicatat sebelum ekspresi kondisi digenerate. Ekspresi kondisi digenerate dan hasilnya dipush ke stack. Instruksi JPC digenerate dengan placeholder untuk alamat akhir loop (WHILE_END). Body loop digenerate, diikuti instruksi JMP kembali ke WHILE_START. Setelah posisi akhir loop diketahui, alamat JPC di-backpatch.

FOR

Konstruksi FOR diterjemahkan secara internal menjadi bentuk setara WHILE. Counter diinisialisasi, kondisi batas diperiksa di setiap iterasi, body dieksekusi, dan counter diinkrementasi. Pola generasi TAC-nya mengikuti pola WHILE dengan tambahan inisialisasi dan inkrementasi counter.

BAB II

PERANCANGAN & IMPLEMENTASI

1. Arsitektur Sistem

Milestone 4 menambahkan dua komponen utama ke dalam pipeline Arion Compiler yang telah dibangun pada milestone-milestone sebelumnya. Pipeline lengkap kini terdiri dari lima tahap yang saling terhubung secara sekuensial:

1. Lexical Analysis (Milestone 1): Membaca source code karakter demi karakter dan menghasilkan daftar token.
2. Syntax Analysis (Milestone 2): Mengubah daftar token menjadi Parse Tree menggunakan algoritma Recursive Descent.
3. Semantic Analysis (Milestone 3): Membangun Decorated AST dan Symbol Table, serta melakukan type checking.
4. Intermediate Code Generation (Milestone 4 – komponen baru): Mengubah Decorated AST menjadi daftar instruksi TAC.
5. Interpretation (Milestone 4 – komponen baru): Mengeksekusi daftar instruksi TAC menggunakan Stack Machine.

2. Struktur Modul Baru

Milestone 4 menambahkan empat file baru ke dalam struktur proyek Arion Compiler:

File	Tanggung Jawab
intermediate_code.hpp	Header bersama: definisi OpcodeType, OprType, struct Instruction, dan interface class IntermediateCodeGenerator. Dikunci bersama di awal sebagai kontrak antar anggota tim.
intermediate_code.cpp	Implementasi IntermediateCodeGenerator: helper emit, backpatch, print, serta generator untuk program structure, expressions, statements, dan control flow.
interpreter.hpp	Header Interpreter: definisi class Interpreter, struct RuntimeError, enum VulnType, dan deklarasi seluruh metode Stack Machine.

interpreter.cpp	Implementasi Interpreter: siklus Fetch-Decode-Execute, handler semua instruksi TAC dan operasi OPR, manajemen bpStack, serta deteksi vulnerability (bonus).
-----------------	---

3. Desain Struct dan Enum

Representasi instruksi TAC diimplementasikan menggunakan struct Instruction yang sederhana namun ekspresif. Struct ini menyimpan tiga informasi: opcode berupa enum OpcodeType, level leksikal (selalu 0 pada milestone ini), dan argumen integer yang maknanya bergantung pada opcode.

OpcodeType adalah enum yang mendefinisikan sembilan opcode: OP_INT, OP_LIT, OP_LOD, OP_STO, OP_OPR, OP_JMP, OP_JPC, OP_CAL, dan OP_RET.

OprType adalah enum yang mendefinisikan empat belas operasi aritmatika, relasional, dan I/O yang diakses melalui instruksi OPR (OPR_NEG hingga OPR_WRTLN).

Pemisahan antara opcode dan operasi OPR menjaga struct Instruction tetap ringkas (hanya tiga field) sambil tetap mendukung empat belas operasi berbeda melalui satu instruksi OPR.

4. Implementasi Intermediate Code Generator (ICG)

4.1. Helper emit dan backpatch

Fungsi emit menerima opcode, level, dan argumen, lalu menambahkan Instruction baru ke dalam vector code dan mengembalikan indeks instruksi tersebut. Indeks ini digunakan untuk keperluan backpatching. Fungsi backpatch menerima indeks baris dan nilai target, lalu memperbarui argumen instruksi di baris tersebut dengan nilai target yang baru.

Fungsi nextAddr mengembalikan indeks instruksi berikutnya yang akan digenerate (sama dengan ukuran vector code saat ini). Fungsi ini digunakan untuk mencatat posisi label sebelum memanggil emit.

4.2. Generator Program Structure

Fungsi genProgram menjadi entry point dari seluruh proses ICG. Fungsi ini memanggil genBlock untuk blok program utama, yang di dalamnya memanggil genDeclPart untuk bagian deklarasi dan genCompound untuk bagian pernyataan. Sebelum memasuki blok utama, ICG memancarkan instruksi INT 0 m di mana m adalah ukuran frame yang dihitung dari btabsize ditambah tiga slot reserved.

Untuk deklarasi prosedur dan fungsi, fungsi `genProcDecl` dan `genFuncDecl` bertanggung jawab menghasilkan struktur frame callee. ICG memancarkan instruksi `CAL` ke baris prosedur saat prosedur dipanggil, dan instruksi `RET` di akhir setiap prosedur atau fungsi.

4.3. Generator Expression & Statements

Generator ekspresi diimplementasikan secara rekursif mengikuti struktur AST. Node `BinOp` menghasilkan TAC dengan pola: generate operand kiri, generate operand kanan, lalu emit `OPR` sesuai operator. Node `UnOp` menghasilkan: generate operand, lalu emit `OPR_NEG` atau `OPR_NOT`. Node literal (`IntLit`, `RealLit`, `BoolLit`) langsung menghasilkan instruksi `LIT 0 v`.

Node `Var` (akses variabel) menghasilkan instruksi `LOD diff a` dimana ;

1. `diff - currentLevel - node→level` (perbedaan level leksikal antara posisi saat ini dan level deklarasi variabel)
2. `A` adalah offset variabel dalam frame, diambil dari field `adr` pada entri tabel simbol (tanpa tambahan 3 slot reserved, karena slot static link, dynamic link, dan return address dikelola oleh interpreter secara terpisah).
3. Node assign menghasilkan: generate ekspresi nilai (yang mendorong hasil ke stack), lalu `STO diff a` dengan `diff` dan `a` yang sama seperti pada `LOD`

Pemanggilan `writeln` dan `write` menghasilkan: generate ekspresi argumen, lalu emit `OPR 0 14 (WRTLN)` atau `OPR 0 13 (WRT)`. Selain itu, implementasi generator pada Arion Compiler juga dilakukan dengan pendekatan pada Decorated AST. Setiap node AST dari jenis ekspresi (`AST_BINOP`, `AST_UNOP`, `AST_VAR`, `AST_INT_LIT`, dan lain-lain) atau pernyataan (`AST_ASSIGN`, `AST_PROC_CALL`) memiliki fungsi generator tersendiri. Fungsi-fungsi ini saling memanggil secara rekursif dengan struktur pohon, dan setiap pemanggilan menghasilkan satu atau lebih instruksi TAC yang ditambahkan ke dalam daftar instruksi global

4.4. Generator Control Flow

Generator `IF-ELSE` bekerja dalam tiga fase. Fase pertama adalah generate ekspresi kondisi. Fase kedua adalah emit `JPC` dengan placeholder (angka 0) dan catat indeksnya sebagai `jpcAddr`. Fase ketiga adalah generate blok `THEN`, emit `JMP` dengan placeholder sebagai `jmpAddr`, backpatch `jpcAddr` dengan `nextAddr`, generate blok `ELSE`, lalu backpatch `jmpAddr` dengan `nextAddr`.

Generator `WHILE` bekerja dalam empat fase. Fase pertama adalah catat `loopStart = nextAddr`. Fase kedua adalah generate ekspresi kondisi. Fase ketiga adalah emit `JPC` dengan placeholder sebagai `jpcAddr`. Fase keempat adalah generate body, emit `JMP 0 loopStart`, lalu backpatch `jpcAddr` dengan `nextAddr`.

Generator FOR secara internal mengonversi konstruk FOR menjadi ekuivalen WHILE: inisialisasi counter, cek kondisi batas, eksekusi body, inkrementasi counter, lompat kembali ke pengecekan kondisi.

Generator REPEAT-UNTIL bekerja dengan cara: catat repeatStart = nextAddr, generate body, generate ekspresi kondisi (UNTIL), emit JPC repeatStart (lompat kembali jika kondisi belum terpenuhi).

5. Implementasi Interpreter (Stack Machine)

5.1. Struktur Data Runtime

Interpreter menyimpan tiga register utama: pc (program counter) yang menunjuk ke instruksi berikutnya, sp (stack pointer) yang menunjuk ke slot teratas stack yang terisi, dan frameDepth yang melacak kedalaman frame aktif untuk deteksi stack overflow.

Memori stack diimplementasikan sebagai vector<StackVal> dengan ukuran maksimum MAX_STACK_SIZE = 100.000 slot. Tipe StackVal adalah long long untuk mendukung deteksi overflow/underflow numerik secara akurat.

bpStack (base pointer stack) adalah vector<int> yang menyimpan indeks awal setiap frame aktif. Saat instruksi INT dieksekusi, indeks base frame baru di-push ke bpStack. Saat instruksi RET dieksekusi, base frame saat ini di-pop dari bpStack.

pendingRetAddr dan pendingDynLink adalah variabel sementara yang diisi oleh instruksi CAL dan dibaca oleh instruksi INT callee untuk mengisi slot Dynamic Link dan Return Address pada frame baru.

5.2. Handler Instruksi

Setiap instruksi TAC memiliki fungsi handler tersendiri yang dipanggil oleh fungsi execute melalui mekanisme switch-case.

- **execINT(m)**: mencatat base = sp+1 ke bpStack, mengalokasikan m slot dengan mengisi nol, memperbarui sp, dan mengisi slot link jika ini bukan frame utama.
- **execLIT(v)**: memanggil push(v) untuk mendorong literal ke stack.
- **xecLOD(a)**: memanggil push(load(a)) untuk membaca nilai dari alamat a dan mendorongnya ke stack.
- **execSTO(a)**: memanggil store(a, pop()) untuk menarik nilai dari stack dan menyimpannya ke alamat a.
- **execOPR(o)**: men-dispatch ke handler operasi yang sesuai berdasarkan nilai o.

- `execJMP(l)`: langsung memperbarui `pc = l`.
- `execJPC(l)`: menarik nilai kondisi dari stack; jika nilainya 0 (false), memperbarui `pc = l`.
- `execCAL(l)`: menyimpan `pc` dan `sp` saat ini ke `pendingRetAddr` dan `pendingDynLink`, lalu memperbarui `pc = l`.
- `execRET()`: men-pop base frame dari `bpStack`, memulihkan `sp` dari Dynamic Link dan `pc` dari Return Address, atau menghentikan program jika `bpStack` kosong setelah pop.

5.3. Handler Operasi OPR

Setiap operasi aritmatika dan relasional mengikuti pola pop-compute-push: menarik satu atau dua operand dari stack, menghitung hasilnya, lalu mendorong hasil tersebut kembali ke stack. Untuk operasi binary (ADD, SUB, MUL, DIV, MOD), operand kedua yang di-pop terlebih dahulu adalah operand kanan (b), dan operand pertama yang di-pop berikutnya adalah operand kiri (a). Hal ini memastikan urutan operasi yang benar sesuai dengan konvensi stack.

Operasi relasional (EQL, NEQ, LSS, GEQ, GTR, LEQ) menghasilkan nilai 1 (true) atau 0 (false) yang didorong ke stack. Hasil ini kemudian digunakan oleh instruksi JPC untuk menentukan arah lompatan. Operasi WRT dan WRTLN menarik satu nilai dari stack dan mencetaknya ke output stream yang dapat dikonfigurasi, memudahkan pengujian unit.

5.4. Deteksi Vulnerability (Bonus)

Interpreter dilengkapi fitur deteksi kerentanan runtime yang dapat diaktifkan melalui fungsi `enableVulnDetection`. Ketika aktif, interpreter memeriksa kondisi-kondisi berikut:

Vulnerability	Kondisi yang Dideteksi
Stack Overflow	Kedalaman frame (<code>frameDepth</code>) melebihi <code>MAX_STACK_DEPTH = 1000</code> , atau <code>sp</code> melebihi <code>MAX_STACK_SIZE</code> .
Stack Underflow	Operasi pop dipanggil ketika <code>sp < 0</code> (stack kosong).
Out-of-Bounds Access	Operasi load atau store mengakses alamat di luar rentang <code>[0, sp]</code> .

Invalid Jump Target	Instruksi JMP, JPC, atau CAL menargetkan baris yang tidak ada (< 0 atau $\geq$ panjang program). Dicek sebelum eksekusi dimulai.
Numerical Overflow	Hasil operasi aritmatika melebihi INT_MAX ($2^{31} - 1$).
Numerical Underflow	Hasil operasi aritmatika kurang dari INT_MIN (-2^{31}).

Semua kondisi vulnerability dilempar sebagai `RuntimeError` dengan pesan yang informatif, mencantumkan jenis vulnerability dan detail konteks (nilai `sp`, alamat yang diakses, baris yang dituju, dan sebagainya).

6. Integrasi

Integrasi kedua komponen baru ke dalam `main.cpp` dilakukan dengan menambahkan mode eksekusi baru. Mode `-i` (interpreter) menjalankan pipeline lengkap: `Lexer` $\rightarrow$ `Parser` $\rightarrow$ `Semantic Analyzer` $\rightarrow$ `ICG` $\rightarrow$ `Interpreter`. Setelah `ICG` menghasilkan daftar instruksi, program mencetak seluruh TAC ke layar atau file output, kemudian `Interpreter` mengeksekusi instruksi tersebut dan mencetak hasil program Arion.

Integrasi ini dirancang agar tidak merusak mode yang sudah ada (`-s` untuk source code dengan output hingga semantic, `-p` untuk parse tree). Mode baru cukup ditambahkan sebagai cabang kondisional baru di `main.cpp` tanpa mengubah alur yang sudah berjalan.

BAB III

PENGUJIAN

1. Test Case 1

1.1. Input:

```
program AljabarSederhana;  
var a, b, c: integer;  
begin  
  a := 5;  
  b := 3;  
  c := a + b * 2;  
  writeln(c);  
end.
```

1.2. Output:

```
--- Intermediate Code Generation ---
```

```
0 INT 0 6
```

```
1 JMP 0 2
```

```
2 LIT 0 5
```

```
3 STO 0 3
```

```
4 LIT 0 3
```

```
5 STO 0 4
```

```
6 LOD 0 3
```

```
7 LOD 0 4
```

```
8 LIT 0 2
```

```
9 OPR 0 4
```

```
10 OPR 0 2
```

```
11 STO 0 5
```

```
12 LOD 0 5
```

```
13 OPR 0 14
```

```
14 RET 0 0
```

```
Output ICG: test/milestone4/1_icg.txt
```

```
--- Execution (Interpreter) ---
```

```
11
```

2. Test Case 2

2.1. Input:

```
program CekGanjilGenap;  
var x, hasil: integer;  
begin  
  x := 15;  
  if (x mod 2) == 0 then  
    hasil := 1  
  else  
    hasil := 0;
```

```
writeln(hasil);  
end.
```

2.2. Output:

```
--- Intermediate Code Generation ---  
0 INT 0 5  
1 JMP 0 2  
2 LIT 0 15  
3 STO 0 3  
4 LOD 0 3  
5 LIT 0 2  
6 OPR 0 6  
7 LIT 0 0  
8 OPR 0 7  
9 JPC 0 13  
10 LIT 0 1  
11 STO 0 4  
12 JMP 0 15  
13 LIT 0 0  
14 STO 0 4  
15 LOD 0 4  
16 OPR 0 14  
17 RET 0 0  
Output ICG: test/milestone4/2_icg.txt  
  
--- Execution (Interpreter) ---  
0
```

3. Test Case 3

3.1. Input:

```
program HitungDeret;  
var n, total: integer;  
begin  
  n := 4;  
  total := 0;  
  while n > 0 do  
    begin  
      total := total + n;  
      n := n - 1;  
    end;  
  writeln(total);  
end.
```

3.2. Output:


```
--- Intermediate Code Generation ---
```

```
0 INT 0 5
```

```
1 JMP 0 2
```

```
2 LIT 0 4
```

```
3 STO 0 3
```

```
4 LIT 0 0
```

```
5 STO 0 4
```

```
6 LOD 0 3
```

```
7 LIT 0 0
```

```
8 OPR 0 11
```

```
9 JPC 0 19
```

```
10 LOD 0 4
```

```
11 LOD 0 3
```

```
12 OPR 0 2
```

```
13 STO 0 4
```

```
14 LOD 0 3
```

```
15 LIT 0 1
```

```
16 OPR 0 3
```

```
17 STO 0 3
```

```
18 JMP 0 6
```

```
19 LOD 0 4
```

```
20 OPR 0 14
```

```
21 RET 0 0
```

```
Output ICG: test/milestone4/3_icg.txt
```

```
--- Execution (Interpreter) ---
```

```
10
```

4. Test Case 4

4.1. Input:

```
program CekRange;  
var x, hasil: integer;  
begin  
  x := 12;  
  if x > 10 then  
    begin  
      if x < 15 then  
        hasil := 1  
      else  
        hasil := 2;  
      end  
    else  
      hasil := 0;  
    writeln(hasil);  
  end.
```

4.2. Output:

```
--- Intermediate Code Generation ---
```

```
0 INT 0 5
```

```
1 JMP 0 2
```

```
2 LIT 0 12
```

```
3 STO 0 3
```

```
4 LOD 0 3
```

```
5 LIT 0 10
```

```
6 OPR 0 11
```

```
7 JPC 0 18
```

```
8 LOD 0 3
```

```
9 LIT 0 15
```

```
10 OPR 0 9
```

```
11 JPC 0 15
```

```
12 LIT 0 1
```

```
13 STO 0 4
```

```
14 JMP 0 17
```

```
15 LIT 0 2
```

```
16 STO 0 4
```

```
17 JMP 0 20
```

```
18 LIT 0 0
```

```
19 STO 0 4
```

```
20 LOD 0 4
```

```
21 OPR 0 14
```

```
22 RET 0 0
```

```
Output ICG: test/milestone4/4_icg.txt
```

```
--- Execution (Interpreter) ---
```

```
1
```

5. Test Case 5

5.1. Input:

```
program TabelPerkalian;
var i, j, total: integer;
begin
  total := 0;
  i := 2;
  while i > 0 do
  begin
    j := 2;
    while j > 0 do
    begin
      total := total + 1;
      j := j - 1;
    end;
    i := i - 1;
  end;
  writeln(total);
end.
```

5.2. Output:



```
--- Intermediate Code Generation ---
0 INT 0 6
1 JMP 0 2
2 LIT 0 0
3 STO 0 5
4 LIT 0 2
5 STO 0 3
6 LOD 0 3
7 LIT 0 0
8 OPR 0 11
9 JPC 0 30
10 LIT 0 2
11 STO 0 4
12 LOD 0 4
13 LIT 0 0
14 OPR 0 11
15 JPC 0 25
16 LOD 0 5
17 LIT 0 1
18 OPR 0 2
19 STO 0 5
20 LOD 0 4
21 LIT 0 1
22 OPR 0 3
23 STO 0 4
24 JMP 0 12
25 LOD 0 3
26 LIT 0 1
27 OPR 0 3
28 STO 0 3
29 JMP 0 6
30 LOD 0 5
31 OPR 0 14
32 RET 0 0
Output ICG: test/milestone4/5_icg.txt
```

```
--- Execution (Interpreter) ---
4
```

6. Test Case 6

6.1. Input:

```
program PrioritasKurang;  
var hasil: integer;  
begin  
  hasil := (100 - 20) div (5 + 3);  
  writeln(hasil);  
end.
```

6.2. Output:

```
--- Intermediate Code Generation ---
```

```
0 INT 0 4
```

```
1 JMP 0 2
```

```
2 LIT 0 100
```

```
3 LIT 0 20
```

```
4 OPR 0 3
```

```
5 LIT 0 5
```

```
6 LIT 0 3
```

```
7 OPR 0 2
```

```
8 OPR 0 5
```

```
9 STO 0 3
```

```
10 LOD 0 3
```

```
11 OPR 0 14
```

```
12 RET 0 0
```

```
Output ICG: test/milestone4/6_icg.txt
```

```
--- Execution (Interpreter) ---
```

```
10
```

7. Test Case 7**7.1. Input:**

```
program CekNegatif;  
var x, hasil: integer;  
begin  
  x := 5;  
  x := -x;  
  if x <= 0 then  
    begin  
      if x != 0 then  
        hasil := 100  
      else  
        hasil := 200;  
      end  
    else  
      hasil := 300;  
    writeln(hasil);  
  end.
```

7.2. Output:



--- Intermediate Code Generation ---

```
0 INT 0 5
1 JMP 0 2
2 LIT 0 5
3 STO 0 3
4 LOD 0 3
5 OPR 0 1
6 STO 0 3
7 LOD 0 3
8 LIT 0 0
9 OPR 0 12
10 JPC 0 21
11 LOD 0 3
12 LIT 0 0
13 OPR 0 8
14 JPC 0 18
15 LIT 0 100
16 STO 0 4
17 JMP 0 20
18 LIT 0 200
19 STO 0 4
20 JMP 0 23
21 LIT 0 300
22 STO 0 4
23 LOD 0 4
24 OPR 0 14
25 RET 0 0
Output ICG: test/milestone4/7_icg.txt
```

--- Execution (Interpreter) ---

100

BAB IV

KESIMPULAN DAN SARAN

1. Kesimpulan

Pada Milestone 4 ini, Arion Compiler berhasil dikembangkan hingga tahap akhir pipeline kompilasi, yaitu Intermediate Code Generation dan Interpretation. Program kini mampu menjalankan source code Arion secara end-to-end: mulai dari tokenisasi, parsing, analisis semantik, pembangkitan Three-Address Code, hingga eksekusi program menggunakan Stack Machine.

Intermediate Code Generator berhasil mengubah Decorated AST hasil Milestone 3 menjadi daftar instruksi TAC yang benar untuk seluruh konstruk bahasa Arion yang didukung, meliputi:

- Ekspresi aritmatika, relasional, dan logika dengan preseden operator yang tepat.
- Pernyataan assignment untuk variabel skalar maupun elemen array.
- Pernyataan output (write dan writeln) untuk berbagai tipe data.
- Konstruk control flow: IF-ELSE, WHILE, FOR, dan REPEAT-UNTIL dengan mekanisme backpatching yang benar.
- Deklarasi dan pemanggilan prosedur serta fungsi dengan manajemen frame multi-level.

Stack Machine Interpreter berhasil mengeksekusi seluruh instruksi TAC dengan benar melalui siklus Fetch-Decode-Execute yang konsisten. Manajemen frame berbasis bpStack memungkinkan penanganan pemanggilan prosedur bersarang yang tepat. Sebagai nilai tambah, Interpreter juga dilengkapi fitur deteksi vulnerability runtime (Stack Overflow, Stack Underflow, Out-of-Bounds Access, Invalid Jump Target, Numerical Overflow, dan Numerical Underflow) yang meningkatkan keamanan dan kemudahan debugging program Arion.

Dengan selesainya Milestone 4 ini, Arion Compiler telah memenuhi seluruh tujuan tugas besar IF2224, yaitu membangun kompiler lengkap dari nol yang mencakup seluruh fase utama kompilasi sesuai teori bahasa formal dan automata.

2. Saran

Meskipun implementasi telah berjalan sesuai spesifikasi, terdapat beberapa aspek yang dapat dikembangkan lebih lanjut untuk meningkatkan kualitas sistem.

Pertama, Intermediate Code yang dihasilkan saat ini belum melalui proses optimasi. Penambahan tahap optimasi ICG, seperti constant folding (menyederhanakan ekspresi konstan saat compile-time), dead code elimination (menghapus kode yang tidak pernah dieksekusi), dan common subexpression elimination (menghindari komputasi ulang ekspresi yang sama), dapat secara signifikan mengurangi jumlah instruksi TAC dan meningkatkan efisiensi eksekusi.

Kedua, Stack Machine saat ini menggunakan model memori flat dengan ukuran tetap (`MAX_STACK_SIZE = 100.000 slot`). Untuk program Arion yang lebih kompleks dengan rekursi dalam atau data struktur besar, pendekatan alokasi memori dinamis yang dapat tumbuh sesuai kebutuhan akan lebih robust dan efisien dalam penggunaan memori.

Ketiga, pesan error runtime yang dihasilkan saat ini belum menyertakan informasi nomor baris source code tempat error terjadi. Dengan menyimpan pemetaan antara baris instruksi TAC dan baris source code selama proses ICG, pesan error dapat dibuat jauh lebih informatif dan memudahkan proses debugging bagi pengguna bahasa Arion.

Keempat, implementasi saat ini belum mendukung tipe data real (floating point) secara penuh dalam Stack Machine, karena StackVal menggunakan tipe long long. Dukungan penuh untuk aritmatika floating point dapat ditambahkan dengan menggunakan union atau reinterpretation cast untuk menyimpan nilai float/double dalam slot StackVal yang sama.

Kelima, fitur deteksi vulnerability runtime yang diimplementasikan sebagai bonus dapat dikembangkan lebih lanjut dengan menambahkan Stack Smashing Detection menggunakan canary value dan Stack Corruption Detection melalui validasi simetri push-pop per frame. Fitur-fitur ini akan membuat Arion Compiler menjadi alat pembelajaran yang lebih lengkap untuk konsep keamanan sistem.

BAB V

LAMPIRAN

1. Link Repository GitHub

Berikut adalah tautan untuk mengakses repository pada platform GitHub:

<https://github.com/Alvacodee/KOI-Tubes-IF2224-2026>

2. Link Workspace Diagram Transisi DFA

Berikut adalah tautan untuk mengakses workspace pada platform draw.io:

<https://drive.google.com/file/d/1RiAAAnoB2hjkNPZm29Pn6T1FA5qK6AiLD/view?usp=sharing>

3. Pembagian Tugas

NAMA	NIM	PEMBAGIAN TUGAS	Persentase Kontribusi
Zahran Alvan Putra Winarko	13524124	Interpreter (Stack Machine) Struct Stack Frame + Manajemen Memori Runtime	25%
Neswa Eka Anggara	13524136	Core structure and subprogram declarations	25%



Daniel Putra Rywandi S.	13524143	ICG Control Flow + Integrasi Main Generator	25%
Muh. Hartawan Haidir	13524147	ICG Expressions & Statements Generator	25%



BAB VI

REFERENSI

1. Edunex ITB. (n.d.). *Formal Language and Automata Theory Materials*. Diakses pada 7 Mei 2026, dari <https://edunex.itb.ac.id/>